https://www.bashHowTo.com

Bash Basics

Presented by IEEE Computer Society at UCF





Overview

C:\WINDOWS\system32\cmd.exe

```
ricky@workshop:~$ cat overview.txt
 -  What is Bash?
 -  What is Bash used for?
 -  Things to know about Bash.
 -  Bash commands
 -  Bash syntax
 -  Bash Behaviors
 -  Bash scripts
ricky@workshop:~$ clear
```

What is Bash?

C:\WINDOWS\system32\cmd.exe

```
ricky@workshop:~$ cd bash && cat
README.txt
  -   Bash (Bourne-Again Shell) is a
      command-line interface (or
      command-line shell)
  -   Command-line Interfaces allow us,
      the user, to interact with software
      by using text commands
  -   It was created in 1989 by Brian Fox
      for the GNU Project
  -   Bash is free and open source (It
      was created as the free software
      alternative to the Bourne shell)
  -   It is the default shell for many
      linux distributions
ricky@workshop:~/bash$ clear
```

GNU - Image

File Edit View Image Colors Help



Image by Victor Siame <vcopovi@wanadoo.fr>, FAL, via Wikimedia Commons

Uses of Bash

C:\WINDOWS\system32\cmd.exe

```
ricky@workshop:~/bash$ cat general.txt

    Bash is Turing-complete so if it is theoretically possible to do some
    computation then it is possible to do it with Bash.

ricky@workshop:~/bash$ cat so_cool.txt

    I will only mention a few of the things that could be done with bash that is
    useful. Since Bash is so versatile there are countless things that you could
    use it for.

ricky@workshop:~/bash$ clear
```

Uses of Bash Continued

C:\WINDOWS\system32\cmd.exe

```
ricky@workshop:~/bash$ cat examples.txt
  -   Servers!
      -> You can make a bash script to automatically handle backups
      -> Script to handle server start-up and restarts
  -   Software Development!
      -> Bash can be used to make install scripts (I actually used one recently)
      -> Bash scripts can be used to automate testing code before pushing updates
  -   Personal use!
      -> You can make a script that will help you clean folders (move certain file
         types into specific places
      -> You can make a script to rename large amounts of files (be aware this
         would be difficult to do if you want individualized names this is for
         well formatted names)
      -> Make a script to log specific data using your terminal (expenses for
         example)
ricky@workshop:~/bash$ clear
```

Things to Know

C:\WINDOWS\system32\cmd.exe

```
ricky@workshop:~/bash$ cat important_{1..3}.txt

 -   Bash can be used in two modes
     -> Interactive - commands are read via input through the terminal
     -> Non-Interactive - commands are read from files (scripts)
 -   Don’t run Bash scripts (or any others) that you don’t understand (this is
     how you accidentally delete important things or worse)
 -   Shell scripts should not be ran as the root user (unless needed). The root
     user has higher privileges and can cause severe problems, like deleting all
     of your files, if not handled carefully
 -   Bash scripts should start with #!/bin/bash or #!/usr/bin/env bash

ricky@workshop:~/bash$ clear
```

Functions

C:\WINDOWS\system32\cmd.exe

```
ricky@workshop:~/bash$ cat builtin.txt

  -   cd - Lets you change directories
  -   echo - Writes arguments to standard
      output (Displays your command to
      the terminal)
  -   for;do;done - repeat a command a
      set amount of times
  -   help - Gives you information about
      Bashs Builtin commands

ricky@workshop:~/bash$ clear
```

C:\WINDOWS\system32\cmd.exe

```
ricky@workshop:~/bash$ cat otherFun.txt

  -   ls - Lists files within a directory
      (not bash but still useful)
  -   rm - remove files or directories
  -   touch - create file or change file
      timestamps
  -   mkdir - make directories
  -   cat - concatenate files and print
      on the standard output
  -   man - interface to show system
      reference manuals

ricky@workshop:~/bash$ clear
```

Syntax

C:\WINDOWS\system32\cmd.exe

```
ricky@workshop:~/bash$ cat syntax.txt

To separate commands bash uses spaces (THIS IS IMPORTANT)

Take the following commands for example

  -   rm -rf /hello
  -   rm -rf / hello

The first command will remove the directory hello and everything in it while the
second one could destroy your entire computer if it is ran as the super user
(sudo). There are protections in place so that this won't actually happen but it
is important to mention.

ricky@workshop:~/bash$ clear
```

Syntax cont

C:\WINDOWS\system32\cmd.exe

```
ricky@workshop:~/bash$ cat syntax.txt

Bash typically works by calling a command and its following arguments (note that
this is a general statement and not the rule)

  -   echo ‘Hello!’

The command is echo and the argument is ‘Hello!’

Lets run this command (and a few others) to see what happens

ricky@workshop:~/bash$ clear
```

Command Test!

C:\WINDOWS\system32\cmd.exe

```
ricky@workshop:~/bash$ echo ‘Hello everyone!’

ricky@workshop:~/bash$ echo “Hello everyone!”

ricky@workshop:~/bash$ echo Hello everyone!

ricky@workshop:~/bash$ cat explain.txt



ricky@workshop:~/bash$ clear
```

Expansions

C:\WINDOWS\system32\cmd.exe

```
ricky@workshop:~/bash$ cat expansion.txt

Expansions are special notations used in bash to essentially substitute certain
patterns in the place of the expansion.

There are 3 main types of expansions I am going to talk about here (there are
others)
  -   Brace expansion
  -   Tilde expansion
  -   Filename expansion

ricky@workshop:~/bash$ clear
```

Expansions Cont.

C:\WINDOWS\system32\cmd.exe

```
ricky@workshop:~/bash$ cat brace_tilde.txt

Brace and tilde expansion are both simple.

Brace expansion is essentially just a sequence. This means that you can create
multiple arguments with increasing or decreasing numbers easily. A command like
“touch file{1..5..1}.txt” would create 5 files from 1 to 5 increasing by 1 each
time. The command “touch file{5..1..-1}.txt” goes from 5 to 1 decrementing by 1
each time. The default is to decrement and increment by 1 so you can just use
{1..5} or {5..1} and it would do the same as the previous ones.

Tilde expansion holds the path to your home directory. So you could go to your
home easily or call a command with files in the home like “cat ~/meow.txt”

ricky@workshop:~/bash$ clear
```

Expansions Cont.

C:\WINDOWS\system32\cmd.exe

```
ricky@workshop:~/bash$ cat file_exp.txt

Filename expansion uses patterns in the place of filenames. There are many of
them but I will go over 3.
  -   Asterisk (zero or more characters)
          *.txt matches all files that end with .txt
  -   Question mark (one character)
          File?.txt matches all files with that form replacing ? with any single
          character
  -   Square Brackets (any one character)
          File[0-9].txt would match to files in that form replacing the bracketed
          space with a single character from 0 to 9

ricky@workshop:~/bash$ clear
```

Variables

C:\WINDOWS\system32\cmd.exe

```
ricky@workshop:~/bash$ cat variables.txt

Variables are just stored chunks of data. If you have experience with any
programming this should be simple. Bash only has untyped variables which means
that you don't have to assign variables a type. Due to the nature of variables
being untyped data (to my knowledge) is all stored as characters and strings.

Variables are very flexible. They can be used to run commands by putting a
command within the variable and expanding that variable.

ricky@workshop:~/bash$ clear
```

Variables

C:\WINDOWS\system32\cmd.exe

```
ricky@workshop:~/bash$ cat variables_2.txt

To create a variable it is easy. Type the variable name and set it equal to what
should be stored. There should be no spaces here.
  -   test=”Hello world!”

To use this data you have to do something called variable expansion. There are a
few ways to do this. To ensure that the expansion is done properly it is
recommended to surround it with double quotes.
  -   echo “$test”
  -   echo $test
  -   echo ${test}
  -   ...

ricky@workshop:~/bash$ clear
```

Positional Parameters

C:\WINDOWS\system32\cmd.exe

```
ricky@workshop:~/bash$ cat arguments.txt

Bash scripts are programs just like these other commands that have been shown.
These programs are all able to take in arguments, including your scripts. The
arguments to your script are the positional parameters and can be accessed very
easily.
  -   echo “$1”
  -   echo “$2”
  -   ... (note that beyond 9 curly braces around the number is needed)
  -   echo “${10}”

You can do all arguments at once by doing “$@” and you can find the number of
arguments given with “$#”

ricky@workshop:~/bash$ clear
```

Conditionals

C:\WINDOWS\system32\cmd.exe

```
ricky@workshop:~/bash$ cat conditionals.txt

Conditionals work similarly to most other languages.
  -   If [[ condition ]] then
          Blah blah blah…
      fi

This is the basic structure of a conditional. The [[ … ]] essentially means to
treat the inside part of the brackets as its own thing, then the if statement
reads the exit code and decides base on that if we need to do what is in the
statement. The fi portion is essentially the end statement.

The [[ ]] isn’t necessary but you would instead have to use a semicolon
otherwise.

ricky@workshop:~/bash$ clear
```

Conditionals Cont.

C:\WINDOWS\system32\cmd.exe

```
ricky@workshop:~/bash$ cat conditionals_2.txt

You can also use else and elif within if statements as well
 -   If [[ condition ]] then
          ...
     elif [[ condition ]] then
          ...
     else
          ...
     fi

These work as you would expect they would if you have experience with
programming. (also remember that the ! operator exist. This will let you invert
expected exit codes. “If ! [[ condition ]] then ...”)

ricky@workshop:~/bash$ clear
```

Conditionals Cont.

C:\WINDOWS\system32\cmd.exe

```
ricky@workshop:~/bash$ cat conditionals_3.txt

For the conditions themselves there are some familiar ones
 ->  String == pattern  (true if they match)
 ->  String != pattern  (true if they don't match)

Then there are less familiar ones which are for file reading
 ->  -e file  (true if the file exists)
 ->  -d file  (true if file exists AND is a directory)
 ->  -f file  (true if file exists AND is a regular file)

These will let you do quite a lot but there are more conditionals that you might
be able to use that I wont be going over. You can also combine conditions using
&&. "if [[ ... ]] && [[ ... ]] ..."

ricky@workshop:~/bash$ clear
```

Loops

C:\WINDOWS\system32\cmd.exe

```
ricky@workshop:~/bash$ cat loops.txt

Loops are useful for many reasons that you are likely already aware of since they
let you run code as many times as you need without having to re-type

Loops work mostly as you would expect and can also be done in.
 -   for i in {1..5} ; do
          ...
     done

For loops can also be run interactively despite what you might think. Bash
recognizes that a loop isn't finished and lets you hit enter without running the
unfinished command.

ricky@workshop:~/bash$ clear
```

Loops Cont.

C:\WINDOWS\system32\cmd.exe

```
ricky@workshop:~/bash$ cat loops_2.txt

Bash also has while loops
  -   while [[ condition ]] ; do
          ...
      done

You can do some weirder things with it as well though
  -   for file in *.pdf ; do
          echo “$file”
      done
  -   for arg in “$@” ; do
          ...
      done

ricky@workshop:~/bash$ clear
```

User Input

C:\WINDOWS\system32\cmd.exe

```
ricky@workshop:~/bash$ cat user_input.txt

It might be useful to be able to take in user input. The command “read” allows
you to read a single line of input. There is more that you can do with this
command than what I will show.

A common use of read where we store input as variable is this...
  -   read -e -p “I am a prompt!” variable

The -e allows the user to let the user be able to use navigation without printing
the escape sequences instead.

The -p is to prompt the user for input.

ricky@workshop:~/bash$ clear
```

Shell Functions

C:\WINDOWS\system32\cmd.exe

```
ricky@workshop:~/bash$ cat functions.txt

Perhaps one of the most important things in scripting is the ability to make
functions within your script. These are called shell functions.
  -   function_name () {
            ...
      }

Since this is important for loops, variables are global by default unless you are
using built-ins that require you to specify otherwise. If you want a local
variable you can use the keyword “local” to make it local.

Note that you can still pass arguments to a shell function.
  -   function_name arg_1 arg_2 ...

ricky@workshop:~/bash$ clear
```

Exit Status

C:\WINDOWS\system32\cmd.exe

```
ricky@workshop:~/bash$ cat exit_status.md

I won't go over this in much detail. All processes return an exit status (0 is
for success). These statuses can represent many different things based on what
the process is doing. These are stored as "$?"

An important thing with statuses though comes in the form of 3 operators &&
(and), || (or), and ! (not). These 3 operators read the exit status and react
according to that.

You can add an exit status to programs by the exit command.

 - exit 9

ricky@workshop:~/bash$ clear
```

Exit Status Cont.

C:\WINDOWS\system32\cmd.exe

```
ricky@workshop:~/bash$ cat exit.txt

As one final thing on Shell functions, you can give shell functions an exit
status or in this case return value. By default a function will have the return
status of its last executed statement.

To set specific an exit status use the return keyword.
  -   return 8

This status is still stored in “$?” for your use.

ricky@workshop:~/bash$ clear
```

And, Or, and Not

C:\WINDOWS\system32\cmd.exe

```
ricky@workshop:~/bash$ cat and_or_not.txt

The && operator executes the first command and only executes the next one if the
first one returns success.

  -   Mkdir “workshop” && cd workshop && touch file.txt

The || operator executes the first command and only executes the next one if the
first one returns failure.

  -   rm file.txt || cat file.txt

The ! operator returns success if the command fails and vice versa.

ricky@workshop:~/bash$ clear
```

Arithmetic

C:\WINDOWS\system32\cmd.exe

```
ricky@workshop:~/bash$ cat arithmetic.txt

You can do addition, subtraction, multiplication, integer division, and modulo.
Double parenthesis means arithmetic so anything else will not work as expected.
  -   echo $(( ... ))
  -   echo $(( ((3 + (4 - 1) * 2) / 2) % 3  ))
          Do these calculations by hand and it is 1.

You can also assign variables and use them in arithmetic. Variables are evaluated
first.
  -   x=5-6
      echo $(( x - 1 ))

ricky@workshop:~/bash$ clear
```

Redirects

C:\WINDOWS\system32\cmd.exe

```
ricky@workshop:~/bash$ cat redirect.txt

An important aspect of shell scripting is the ability to redirect input and
output.

There are 3 main data streams (file descriptors) to worry about.
  -   Standard Input (File Descriptor 0)
  -   Standard Output (File Descriptor 1)
  -   Standard error (File Descriptor 2)

The data sent down these streams can be connected and sent to new files if you
want. You can also send this data into other commands as parameters. The file
descriptor is part of how you specify the stream to redirect.

ricky@workshop:~/bash$ clear
```

Redirects Cont.

C:\WINDOWS\system32\cmd.exe

```
ricky@workshop:~/bash$ cat redirect-output_1.txt

Redirecting output can be viewed as “left to right”

An example would be “cat file.txt > out.txt” where you call the cat command on
file.txt and then overwrite the file out.txt with the output of the command.

The command “cat file.txt >> out.txt” does almost the exact same thing with one
big change. The file out.txt is not overwritten and is instead appended to.

This however redirects only the Standard Out but you might need to redirect the
Standard Error as well.

ricky@workshop:~/bash$ clear
```

Redirects Cont.

C:\WINDOWS\system32\cmd.exe

```
ricky@workshop:~/bash$ cat redirect-output_2.txt

To redirect multiple data streams there are 2 easy ways (at least that I know)
  -   cat file.txt &>> out.txt
          This makes assumptions about what you want. It assumes you want to
          redirect Standard Out (FD 1) and Standard Error (FD 2).
  -   cat file.txt >> out.txt 2>&1
          This lets you specify what you redirect. Essentially what is being done
          here is you are connecting FD 2 to the same stream as FD 1.

You can also redirect each data stream individually.
  -   cat file.txt >> out.txt 2>> error.txt
  -   cat file.txt 2>> error.txt

ricky@workshop:~/bash$ clear
```

Redirects Cont.

C:\WINDOWS\system32\cmd.exe

```
ricky@workshop:~/bash$ cat redirect-input.txt

Redirecting input can be viewed as “right to left”

An example would be “rm -rf < in.txt” where you call the rm command using what is
inside in.txt as input. The uses for this is a tad more niche than output
redirection.

Simply put this is mostly just a way that you can input large amounts of data
into a command without typing it all out in the command line. Specifically if you
make a program that you wish to run multiple times with large amounts of inputs
this could be useful.

As a reminder Standard input is File descriptor 0 (FD 0).
ricky@workshop:~/bash$ clear
```

Redirects Cont.

C:\WINDOWS\system32\cmd.exe

```
ricky@workshop:~/bash$ cat pipelines.txt

Pipelines are essentially just a specific type of redirect. They let you execute
a command and then send the output of that command as the input for another.

  -   cat file.txt | grep “Hello”

ricky@workshop:~/bash$ clear
```

Conclusion

C:\WINDOWS\system32\cmd.exe

```
ricky@workshop:~/bash$ cat conclusion.txt

Before I finish this workshop we will make a very quick bash script. Feel free to
leave if you would like. I will give you a minute before we start.
```

